Digital Hardware Design Assignment

Verilog codes and Simulations

Teja Krishna.K

Teja krishna Kopuri
1700363C204
ECE

INDEX:

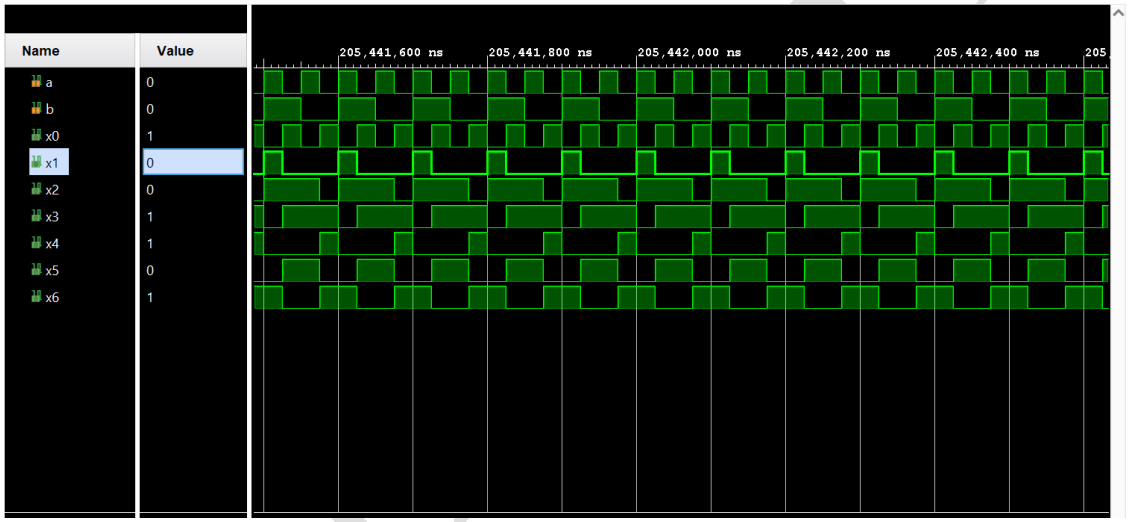
1. All gates
2. Conversions
3. Mux
4. Flip Flops
5. Shift registers
6. All gates using task
7. Counters
8. Decoder
9. encoder
10. Time delay exercise

Teja Krishna.K

1. All gates

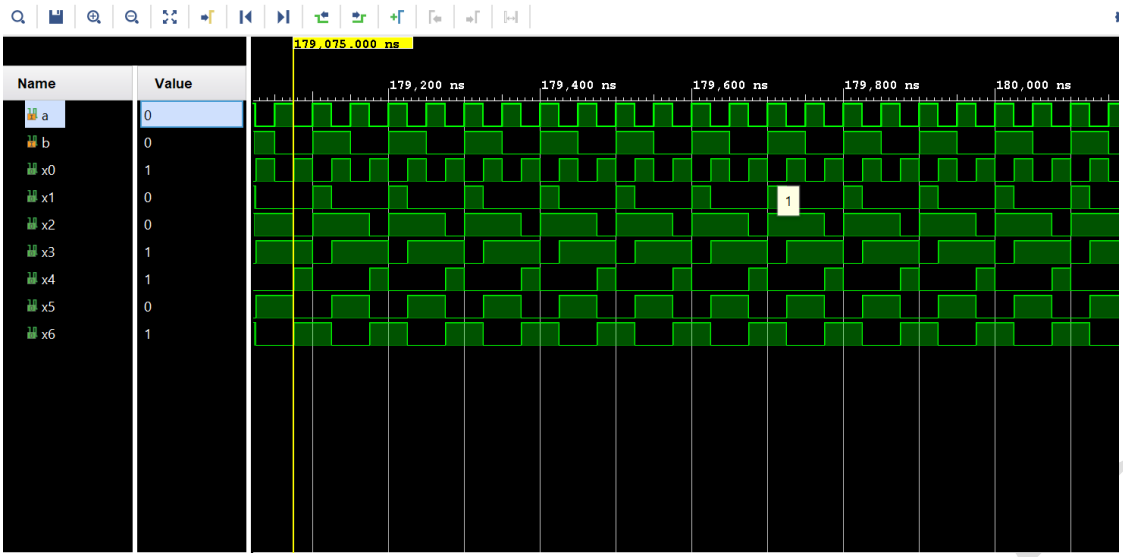
Gate level:

```
Module all_gates(
input a,b,
output x0,x1,x2,x3,x4,x5,x6,);
not n1(x0,a);
and a1(x1,a,b);
or o1(x2,a,b);
nand na1(x3,a,b);
nor no1(x4,a,b);
xor x1(x5,a,b);
xnor xn1(x6,a,b);
endmodule
```



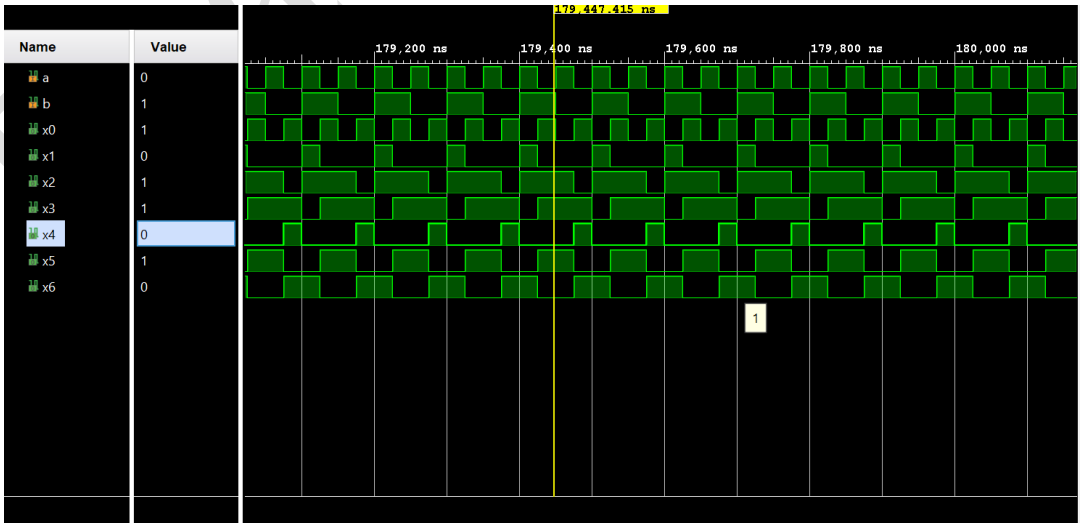
Data flow:

```
Module all_gates(
input a,b,
output x0,x1,x2,x3,x4,x5,x6,);
assign x0=~a;
assign x1=a&b;
assign x2=a|b;
assign x3=~(a&b);
assign x4=~(a|b);
assign x5=a^b;
assign x6=~(a^b);
endmodule
```



Behavioural modelling:

```
Module all_gates(
Input a,b,
Output x0,x1,x2,x3,x4,x5,x6,);
always@(*)
begin
x0=~a;
x1= a&b;
x2=a|b;
x3=~(a&b);
x4=~(a|b);
x5=a^b;
x6=~(a^b);
end
endmodule
```



2. Conversions:

1. Binary to Grey:

```
module B_TO_G(input [3:0]b,
output [3:0]g);
assign g[0]=b[0];
assign g[1]=b[0]^b[1];
assign g[2]=b[1]^b[2];
assign g[3]=b[2]^b[3];
endmodule
```

2. Grey to Binary:

```
module G_TO_B(input [3:0] g,
output [3:0] b);
assign b[0]= g[0];
assign b[1]=b[0]^g[1];
assign b[2]=b[1]^g[2];
assign b[3]=b[2]^g[3];
```

3. Mux

1. 4x1 Mux

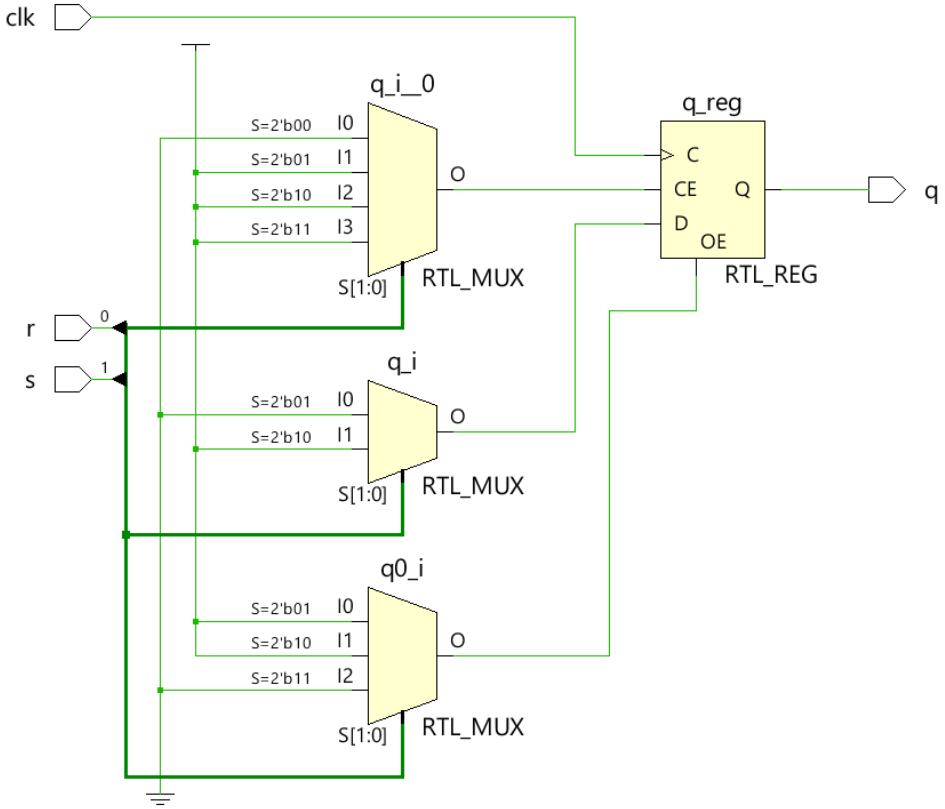
```
module 4x1_mux(
     input [3:0] a;
     input [2:0] sel;
     output out;
     always@(a,sel)
begin
     case(sel)
     2'b00:out=a[0];
     2'b01:out=a[1];
     2'b10:out=a[2];
     2'b11:out=a[3];
    endcase
end
endmodule
```

Teja Krishna.K

4. Flip Flops

4.1 SR Flip Flop:

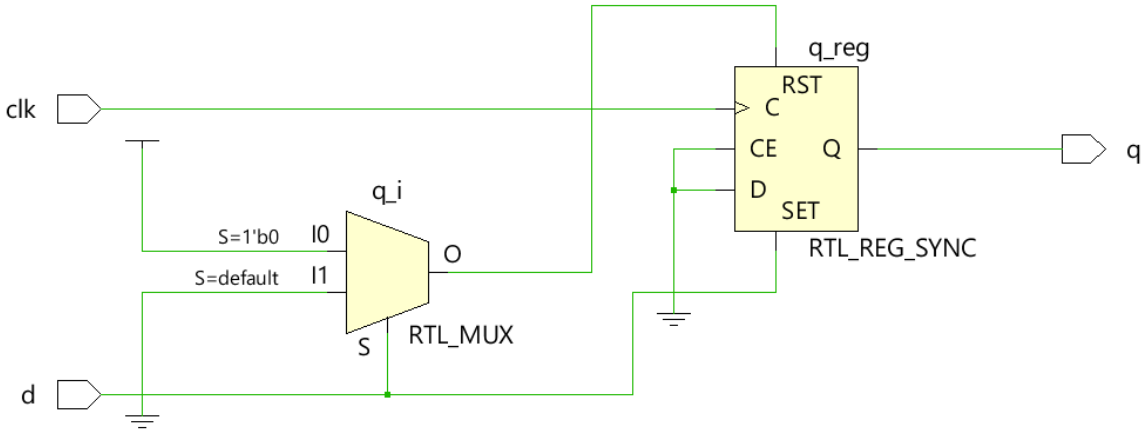
```
module SR_FF(input s,r,clk, output reg q);
always@(posedge clk)
begin
   case({s,r})
    2'b00:q<=q;// remains in previous state
    2'b01:q<=0;//FF will reset
    2'b10:q<=1;//FF will set
    2'b11:q<=1'bz;//Forbidden state
endcase
end
endmodule
```



4.2 D Flip Flop

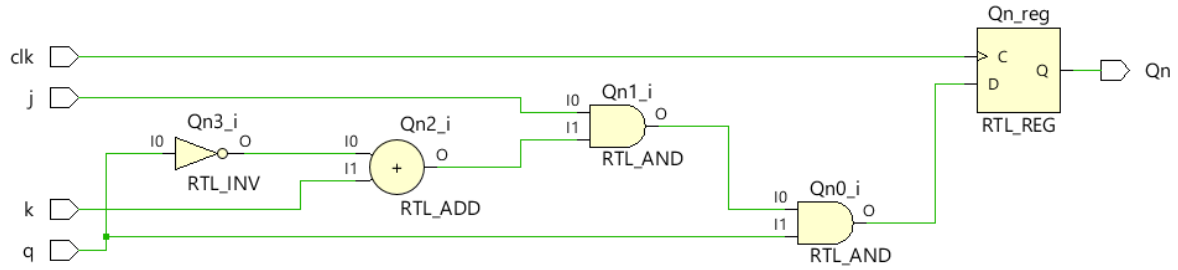
```
module DFF(input clk, d,rst, output reg q);
always@(posedge clk or posedge rst)
begin
  if(rst)
  q<=0;
  else
  q<=d;

end
endmodule
```



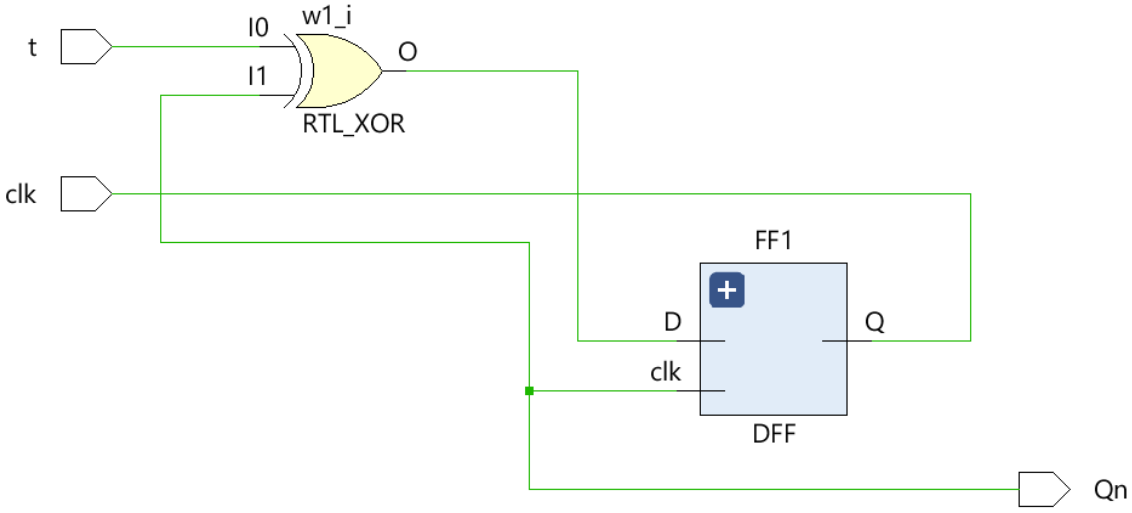
4.3 JK Flip Flop:

```
module JK_FF( input j,k,clk,q, output reg Qn);
always@(posedge clk)
begin
  Qn<=(j&(~q)+k&q);// using characteristic equation
end
endmodule
```



4.4 T Flip Flop:

```
module T_FF(input t,clk, output  Qn);
  wire w1,w2;
  assign w1=t^w2;
  D_FF FF1(w2,w1,clk);// Instance method
  assign Qn=w2;
endmodule
```



5. Shift Registers:

5.1 SISO(serial in serial out):

```
module SISO(input clk,rst,si, output reg so);
reg [3:0] tmp;
always@(posedge clk, negedge rst)
begin
  if(!rst)//rst==0
  tmp=4'b0;
  else
begin
  tmp<=tmp>>1;// shift right 1 bit
  tmp[3]<=si;// Assigning
  so= tmp[0];
end
end
endmodule
```

5.2 SIPO(Serial In Parallel Out):

```
module SIPO(input clk,rst,si, output reg [3:0] po);
reg[3:0] tmp;
always@(posedge clk, negedge rst)
begin
  if(!rst)
  tmp<=4'b0000;
  else
  tmp<=tmp<<1;
  tmp[0]<=si;
  po= tmp;
end endmodule
```

Teja Krishna..

5.3 PISO(Parallel In serial Out):

```
module PISO(input [3:0] pi,input clk, rst,output reg so);
reg [3:0]tmp;
always@(posedge clk)
begin
  if(!rst)
begin
  so<=1'b0;
end
  else
  begin
  tmp<=tmp>>1;
  tmp<=pi;
  so<=tmp[0];
end
end
endmodule
```

5.4 PIPO(Parallel In Parallel Out):

```
module PIPO(input [3:0] pi, input clk, rst,output reg [3:0] po);
reg [3:0]tmp;
always@(posedge clk)
begin
  if(!rst)
begin
  po<=4'b0;
end
  else
begin
  tmp<=pi;
  po<=tmp;
end
end
endmodule
```

Teja Krishna.K

6. All gates using task:

```
module task_00(
    input a,
    input b
    );
    reg and_gate, or_gate, xor_gate, nand_gate, nor_gate, xnor_gate;

    task all_gates;
    input a,b;
    output and_gate, or_gate, xor_gate, nand_gate, nor_gate, xnor_gate;
    begin
    #20 and_gate = a&b;
    #20 or_gate = a|b;
    #20 xor_gate = a^b;
    #20 nand_gate = ~(a&b);
    #20 nor_gate = ~(a|b);
    #20 xnor_gate = ~(a^b);
    end
    endtask
    always@(a,b)
    begin
    all_gates(a,b,and_gate, or_gate, xor_gate, nand_gate, nor_gate,
xnor_gate);
    end
endmodule
```

7. Counters:

7.1 mod 15 counter:

```
module mod_N_counter(input clk,rst,output reg [3:0] C_out);// mod 15
counter
always@( posedge clk)
begin
if(rst)
C_out<=4'b0000;
else if(C_out==4'b1111)
C_out<=4'b0000;
else
C_out<= C_out+1;
end
endmodule
```

Teja Krishna.K

8. Decoder

8.1 2x4 decoders:

```
module decoder2x4(input a,b,
 output I0,I1,I2,I3);
 assign I0=((~a)&(~b));
 assign I1=((~a)&(b));
 assign I2=((a)&(~b));
 assign I3=((a)&(b));
 endmodule
```

9. Encoder

9.1 4x2 Encoder

```
 module encoder4_2 ( din ,dout );

output [1:0] dout ;
reg [1:0] dout ;

input [3:0] din ;
wire [3:0] din ;

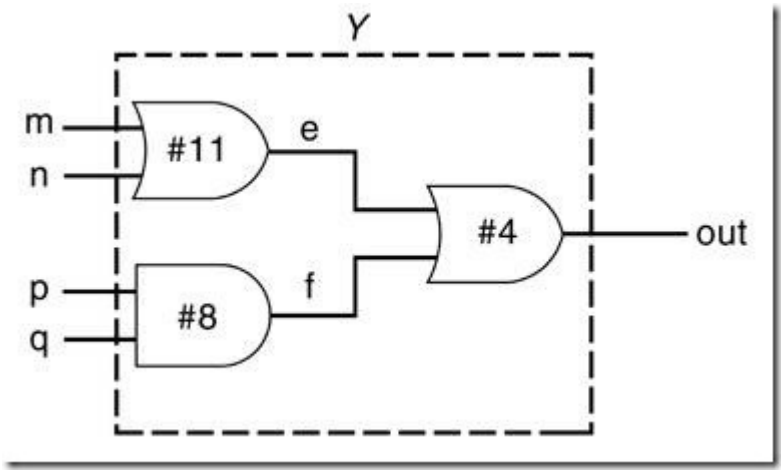
always @ (din) begin
 if (din==8)
  dout = 0;
 else if (din==4)
  dout = 1;
 else if (din==2)
  dout = 2;
 else
  dout = 3;
end

endmodule
```

10. Timing delays:

10.1: What type of delay model is used in the following circuit? Write the Verilog description for the module Y**.**

Teja Krishna.K



Sol: a. Distributed delay

```
module Y(out,m,n,p,q);
input m,n,p,q;
output out;
wire e,f;
or #11 o1(e,m,n);
and #8 a1(f,p,q);
or #4 o2(out,e,f);
endmodule
module test();// test bench
reg m,n,p,q;
wire out;
Y y1(out,m,n,p,q);
initial
begin
$monitor($time,"out=%b,m=%b,n=%b,p=%b,q=%B",out,m,n,p,q);
m=1;n=1;p=1;q=1;
#30 m=1;n=1;p=1;q=1;
#30 m=0;n=1;p=1;q=0;
#30 m=0;n=1;p=0;q=1;
#30 m=0;n=0;p=1;q=1;
end
endmodule
add wave -r /*
run
#0out=x,m=1,n=1,p=1,q=1
#12out=1,m=1,n=1,p=1,q=1
#60out=1,m=0,n=1,p=1,q=0
#90out=1,m=0,n=1,p=0,q=1
```

10.2 Use the largest delay in the module to convert the circuit to a lumped delay model. Using a lumped delay model, write the Verilog description for the module Y.

Sol:

Teja Krishna.K

```
module Y_2(out,m,n,p,q);
input m,n,p,q;
output out;
wire e,f;
or  o1(e,m,n);
and a1(f,p,q);
or #15 o2(out,e,f);
endmodule
```

10.3 Compute the delays along each path from input to output for the circuit in Exercise 1. Write the Verilog description, using the path delay model. Use specify blocks.

Sol:

```
module Y_3(out,m,n,p,q);
input m,n,p,q;
output out;
wire e,f;
specify
(m => out)=15;
(n => out)=15;
(p => out)=12;
(q => out)=12;
endspecify
or  o1(e,m,n);
and  a1(f,p,q);
or o2(out,e,f);
endmodule
```

10.4 Consider the negative edge-triggered with the asynchronous reset D-flipflop shown in the figure below. Write the Verilog description for the module D_FF. Show only the I/O ports and path delay specification. Describe path delays, using parallel connection.

Sol:

```
module DFF_DELAYS_4( input d,clk,rst, output reg q,qbar);
 specify
 (d =>q)=5;
 (d =>qbar)=5;
 (clk=> q)=6;
 (clk=>qbar)=7;
 (rst=>q)=3;
 (rst=>qbar)=4;
endspecify
always@(negedge clk or posedge rst)
begin
 if(rst)
  begin
   q<= 0;
   qbar<=1;
  end
```

Teja Krishna.K

```
 else
  begin
   q<=d;
   qbar<=~d;
  end
 end
endmodule
```

10.5 Modify the D-flipflop in Exercise 4 if all path dealys are 5 units. Describe the path delays, using full connections to q and qbar.

Sol:

```
module DFF_DELAYS_5( input d,clk,rst, output reg q,qbar);
 specify
 (d,clk,rst*>q,qbar)=5;
endspecify
always@(negedge clk or posedge rst)
begin
 if(rst)
  begin
   q<= 0;
   qbar<=1;
  end
 else
  begin
   q<=d;
   qbar<=~d;
  end
 end
endmodule
```

10.6 Assume that a six-delay specification is to be specified for all path delays. All path delays are equal. In the specify block, define parameters t_01=4,t_10=5,t_0z=7,t_z1=2,t_1z=3,t_z0=8. Use the D-flipflop in Exercise 4 and write the six-delay specification for all paths, using full connections.

Sol:

```
module DFF_DELAYS_6( input d,clk,rst, output reg q,qbar);
specify
specparam t_01=5,t_10=4,t_0z=6,t_z0=5,t_1z=5,t_z1=6;
(d,clk,rst*>q,qbar)=(t_01,t_10,t_0z,t_z0,t_1z,t_z1);
endspecify
always@(negedge clk or posedge rst)
begin
if(rst)
begin
```

Teja Krishna.K

```
q<= 0;
qbar<=1;
end
else
begin
q<=d;
qbar<=~d;
end
end
endmodule
```

10.7 In Exercise 4,modify the delay specification for the D-flipflop if the delays are dependent on the value of d as follows:
clock -> q = 5 for d = 1'b0, clock -> q= 6 otherwise
clock -> qbar = 4 for d = 1'b0, clock ->qbar = 7 otherwise
All other delays are 5 units.
Sol:

```
module DFF_DELAYS_7( input d,clk,rst, output reg q,qbar);
specify
if(d) (clk=>q)=5;
if(~d)(clk=>q)=6;
if(d) (clk=>qbar)=5;
if(~d)(clk=>qbar)=6;
(d,clk,rst*>q,qbar)=5;
endspecify
always@(negedge clk or posedge rst)
begin
if(rst)
begin
q<= 0;
qbar<=1;
end
else
begin
q<=d;
qbar<=~d;
end
end
endmodule
```

10.8 For the D-flipflop in Exercise 7, add timing checks for the D_flipflop in the specify block as follows:
The minimum setup time for d with respect to clock is 8.
The minimum hold time for d with respect to clock is 4.
The reset signal is active high. The minimum width of a reset pulse is 32.

Teja Krishna.K

Sol:

```
module DFF_DELAYS_8( input d,clk,rst, output reg q,qbar);
specify
$setup(d,clk,8);
$hold(clk,d,4);
$width( posedge rst,32);
endspecify
always@(negedge clk or posedge rst)
begin
if(rst)
begin
q<= 0;
qbar<=1;
end
else
begin
q<=d;
qbar<=~d;
end
end
endmodule
```

Teja Krishna.K